Guia completo: Docker + Node.js do zero (Atividade DevOps)

Nov 9, 2026 · @Gabryel

Este guia leva você de uma pasta vazia até um app Node.js rodando dentro de um contêiner Docker, nas versões 1.0 e 2.0. Cada passo explica o porquê de cada comando e mostra os erros reais que apareceram ao fazer a atividade, com a correção de cada um.

Antes de começar

Conceitos que você precisa ter na cabeça

- **Node.js** é um programa que executa JavaScript fora do navegador, direto no computador. Você roda um arquivo com `node arquivo.js`.
- **Servidor HTTP** é um programa que fica ligado numa porta esperando pedidos (*requests*) e devolvendo respostas (*responses*). A porta é como o número de um apartamento: o computador é o prédio e cada programa atende num número diferente.
- **Docker** empacota um aplicativo com tudo que ele precisa para rodar, para que funcione igual em qualquer máquina.
- **Imagem** é o pacote fechado, só para leitura. Funciona como um molde e é criada com `docker build`.
- **Contêiner** é a imagem em execução, criada com `docker run`. Da mesma imagem podem sair vários contêineres.
- **Dockerfile** é o arquivo de texto com a receita para construir a imagem.

Pré-requisitos

1. Node.js instalado (serve só para testar o app antes de usar o Docker).
2. Docker Desktop instalado **e aberto**.
3. Um editor de texto, como o VS Code.
4. O PowerShell ou outro terminal.

Confira tudo no terminal

```
node -v
docker --version
docker info
```

Os dois primeiros devem mostrar um número de versão. O `docker info` deve mostrar uma lista longa de informações. Se ele der erro, o Docker Desktop não está rodando: veja o erro do Passo 4.

Pasta do projeto

Crie uma pasta para o projeto. Neste guia ela é `C:\aluno-app`, e todos os comandos são rodados **dentro dela**. Para entrar nela pelo terminal:

```
cd C:\aluno-app
```

Passo 1: criar o app Node.js

O app é um servidor web de poucas linhas que responde um texto a quem acessar `http://localhost:3000`. O Node.js já traz o módulo `http`, então não precisa instalar nada.

1. Crie o arquivo. Dentro da pasta `aluno-app`, crie o arquivo `app.js`.

2. Entenda as quatro peças do código.

1. **Importar o módulo `http`.** `require('http')` traz as ferramentas de servidor web. `const http = ...` guarda essas ferramentas numa variável que não muda.
2. **Criar o servidor.** `http.createServer(...)` recebe uma função que o Node executa **a cada acesso**. Ela tem dois parâmetros: `req` (o pedido que chegou) e `res` (a resposta que você vai enviar).
3. **Responder.** `res.end('texto')` envia o texto ao navegador e encerra a resposta. Essa linha fica **dentro** da função da peça 2.
4. **Ligar o servidor.** `servidor.listen(3000)` faz o servidor ficar escutando na porta 3000. Fica **fora** da função, no fim do arquivo.

3. O código final (use o seu nome no texto):

```
const http = require('http');

const servidor = http.createServer((req, res) => {
    res.end('Renatao');
});

servidor.listen(3000);
```

4. Teste antes do Docker. No terminal, dentro da pasta:

```
node app.js
```

O terminal fica preso, e isso é normal: o servidor está rodando. Abra `http://localhost:3000` no navegador e o texto deve aparecer. Para parar, use `Ctrl+C`.

Erro comum: se aparecer `EADDRINUSE`, algo já está usando a porta 3000, como um `node app.js` esquecido em outro terminal. Feche-o com `Ctrl+C` e tente de novo.

Passo 2: criar o Dockerfile

O Dockerfile é a receita que o Docker lê de cima para baixo para montar a imagem. Cada linha começa com uma instrução em maiúsculas.

Crie, na mesma pasta do `app.js`, um arquivo chamado exatamente `Dockerfile`: D maiúsculo e **sem extensão** (nada de `.txt`). No Windows, confira se o editor não salvou como `Dockerfile.txt`.

As cinco instruções

1. **`FROM`** define a imagem base, que já existe no Docker Hub. Você precisa de uma com Node.js instalado. O formato é `nome:tag`, e a tag é a versão. `node:20-alpine` é o Node 20 sobre o Alpine, um Linux muito pequeno. Use sempre uma tag específica, nunca `latest`.
2. **`WORKDIR`** define a pasta de trabalho **dentro do contêiner**. Se ela não existir, o Docker cria. As instruções seguintes rodam nela.
3. **`COPY`** copia arquivos da sua máquina (origem) para dentro da imagem (destino). O destino `.` significa a pasta de trabalho atual, a do `WORKDIR`.
4. **`EXPOSE`** documenta qual porta o app usa. Não abre a porta de verdade: quem faz isso é o `-p` no `docker run`.
5. **`CMD`** define o comando que roda quando o contêiner inicia. Cada pedaço vai entre aspas duplas, separado por vírgula, dentro de colchetes.

O Dockerfile final

```
FROM node:20-alpine
WORKDIR /aluno-app
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]
```

O nome da pasta no `WORKDIR` pode ser qualquer um (`/app` é o mais comum). O comando do `CMD` é o mesmo `node app.js` que você digitava no terminal, só que escrito em pedaços.

Cuidados

- Aspas duplas `"` no `CMD`, nunca aspas simples.
- Não deixe espaços sobrando no fim das linhas. Não quebra nada, mas vale o hábito.
- A ordem importa: o `COPY` só funciona depois de o `WORKDIR` definir a pasta de destino.

Passo 3: criar o .dockerignore

O `.dockerignore` lista o que o Docker deve **ignorar** ao montar a imagem. Ele deixa o build mais rápido, a imagem menor e mais segura.

Por que ele existe. Ao rodar `docker build`, o Docker envia toda a pasta do projeto para o motor que faz o build. Isso se chama **contexto de build**. A pasta pode ter coisas que não devem ir para a imagem:

- `node_modules`: pasta enorme de dependências, que deve ser instalada dentro da imagem e não copiada.
- `.git`: histórico do controle de versão.
- `.env`: arquivos com senhas e chaves, que nunca podem ir parar numa imagem.
- Arquivos de log e anotações.

Como criar. Um arquivo de texto chamado exatamente `.dockerignore`, com o ponto no começo e sem extensão, na mesma pasta do `Dockerfile`. Cada linha é um arquivo ou pasta a ignorar. O `*` vale por qualquer coisa, então `*.log` ignora todos os arquivos que terminam em `.log`.

```
node_modules
.git
.env
*.log
Dockerfile
.dockerignore
```

O projeto desta atividade ainda não tem quase nenhum desses arquivos, e o `COPY app.js .` já copia só o `app.js`. O `.dockerignore` fica para criar o hábito e porque a atividade pede.

Dica no Windows. Se o editor insistir em salvar como `.dockerignore.txt`, use o VS Code ou crie pelo terminal. Confira se o nome ficou certo com `dir /a` no `cmd` ou `Get-ChildItem -Force` no PowerShell.

Passo 4: construir a imagem (build)

O `docker build` lê o Dockerfile e cria a imagem `aluno-app` na versão `1.0`. Antes, **abra o Docker Desktop** e espere aparecer o motor rodando (Engine running, com indicador verde).

O comando

```
docker build -t aluno-app:1.0 .
```

Peça por peça:

- `docker build` manda construir uma imagem a partir de um Dockerfile.
- `-t` (de *tag*) dá um nome à imagem que **você está criando**.
- `aluno-app:1.0` é o formato `nome:versão`. A atividade pede o nome `aluno-app` na versão `1.0`.
- O `.` no final diz que o contexto de build é a pasta atual. Esquecer esse ponto é um erro muito comum.

O que aparece. Várias etapas numeradas, uma para cada linha do Dockerfile (`FROM`, `WORKDIR`, `COPY`...). Na primeira vez o Docker baixa a imagem `node:20-alpine` do Docker Hub, e isso pode demorar. No final aparece uma linha `naming to docker.io/library/aluno-app:1.0`.

Confira que a imagem existe:

```
docker images
```

Deve aparecer uma linha com `aluno-app` e a tag `1.0`.

Erro 1: o Docker não está rodando

```
ERROR: failed to connect to the docker API at
npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and
if the daemon is running
```

Causa. O Docker tem duas partes: o **cliente** (o comando `docker` que você digita) e o **daemon** (o motor que trabalha em segundo plano). No Windows, o Docker Desktop liga o motor. O comando existia, mas o motor estava desligado.

Correção.

1. Abra o Docker Desktop pelo menu Iniciar.
2. Espere o motor subir (pode levar um ou dois minutos na primeira vez).
3. Confirme com `docker info`: se aparecer uma lista longa, sem erro, está pronto.

4. Rode o `docker build` de novo.

Se o Docker Desktop abrir mas mostrar erro de WSL ou de virtualização, o problema é uma configuração do Windows, não do seu projeto.

Erro 2: usar o nome da imagem base no `-t`

O comando digitado foi:

```
docker build -t node:20-alpine .
```

Causa. O `-t` define o nome da imagem que você **cria**, e foi colocado o nome da imagem **base**, a do `FROM`. Isso criaria a sua imagem com o nome de outra, e confundiria tudo.

Correção. Use o nome do seu projeto:

```
docker build -t aluno-app:1.0 .
```

Guarde esta diferença:

Onde	Para que serve	Valor
`FROM` no Dockerfile	A imagem base que você **usa**	`node:20-alpine`
`-t` no `docker build`	A imagem que você **cria**	`aluno-app:1.0`

Passo 5: executar o contêiner (docker run)

Até aqui você só tem a imagem, que fica parada. O `docker run` cria um **contêiner** a partir dela e o coloca para funcionar.

O comando

```
docker run -d --name aluno-app-container -p 3000:3000 aluno-app:1.0
```

O formato geral é `docker run [opções] imagem:tag`. As opções vêm **antes** do nome da imagem.

- **`-d`** (*detached*) roda o contêiner em segundo plano. Sem ele, o terminal fica preso esperando.
- **`--name aluno-app-container`** dá um nome ao contêiner. Sem ele, o Docker inventa um nome aleatório, como `quirky_turing`.
- **`-p 3000:3000`** publica a porta, ligando uma porta do seu PC a uma do contêiner.
- **`aluno-app:1.0`** é a imagem que serve de molde, sempre por último.

Entendendo o `-p`. O contêiner é como um prédio fechado: dentro dele o app atende no apartamento 3000, mas o seu navegador não tem entrada. O `-p` instala uma porta de entrada do seu computador até lá dentro:

```
-p  PORTA_NO_SEU_PC : PORTA_NO_CONTEINER
        |                   |
   onde você acessa     onde o app escuta
   no navegador         (dentro do contêiner)
```

O número da direita vem do `servidor.listen(3000)` do `app.js`. O da esquerda é o que você digita no navegador. Um exemplo: `-p 8080:80` significa o que chegar na porta 8080 do PC vai para a porta 80 do contêiner.

O que deve acontecer. O Docker imprime uma sequência longa de letras e números, o ID do contêiner. Abra `http://localhost:3000` e o texto do `app.js` aparece, agora vindo de dentro do contêiner.

Erro 3: `invalid reference format`

```
docker run -d --name aluno-app:1.0 -p 3000:3000 node:20-alpine:1.0
docker: invalid reference format
```

Causa. No final do comando foi escrito `node:20-alpine:1.0`, juntando dois nomes numa coisa só. Uma imagem só pode ter **um** `:`, no formato `nome:tag`.

Correção. Escreva só a sua imagem no final: `aluno-app:1.0`.

Erro 4: usar a imagem base em vez da sua

```
docker run -d --name aluno-app:1.0 -p 3000:3000 node:20-alpine
Unable to find image 'node:20-alpine' locally
```

Causa. `node:20-alpine` é a imagem **base**, a do `FROM`. Por isso o Docker a baixou do Docker Hub. Não é grave (a imagem só ficou guardada), mas você estava rodando o Node puro e não o seu app.

Correção. Rode a imagem que você construiu: `aluno-app:1.0`.

Erro 5: `invalid container name`

```
docker run -d --name aluno-app:1.0 -p 3000:3000 aluno-app:1.0
docker: Error response from daemon: invalid container name (aluno-app:1.0):
only [a-zA-Z0-9][a-zA-Z0-9_.-] are allowed
```

Causa. O nome do **contêiner** só aceita letras, números, `_`, `.` e `-`. O `:` não é permitido. Ele apareceu porque o nome da imagem foi usado no lugar do nome do contêiner.

Correção. Troque só o valor depois de `--name` por um nome inventado, sem `:`, como `aluno-app-container`.

A raiz dos três erros é uma só: **misturar o nome do contêiner com o nome da imagem.**

	O que é	Exemplo
Imagem	O molde que você construiu. Tem versão.	`aluno-app:1.0`
Contêiner	O programa rodando. É um nome inventado por você, sem versão.	`aluno-app-container`

Passo 6: verificar

A atividade pede dois itens: listar os contêineres em execução e listar as saídas do sistema (os logs).

a) Listar os contêineres em execução

```
docker ps
```

Aparecem o ID, a imagem, o comando, o tempo ativo, as portas e o nome. Na coluna `PORTS` deve aparecer algo como `0.0.0.0:3000->3000/tcp`, que é a ponte de portas do `-p` escrita de outro jeito.

Para ver também os contêineres **parados**, use o `-a` (*all*):

```
docker ps -a
```

b) Listar as saídas do sistema (logs)

```
docker logs aluno-app-container
```

Isso mostra tudo que o app escreveu na saída. Na versão 1.0 o `app.js` não escreve nada no console, então o resultado vem **vazio**, e isso é normal. Na versão 2.0 você vai adicionar uma mensagem para ter o que mostrar.

c) Listar as imagens

```
docker images
```

Mostra as imagens que existem no seu computador, com nome, tag e tamanho. Aqui devem aparecer `aluno-app` e `node` (a imagem base que o Docker baixou).

Dica para a entrega. Tire prints do terminal com `docker ps`, `docker logs` e `docker images`, e do navegador mostrando o app. Caso o professor peça evidências, elas já estarão prontas.

Passo 7: criar a versão 2.0

Criar uma versão 2.0 significa **mudar o app e gerar uma nova imagem com outra tag**. Você fica com `aluno-app:1.0` e `aluno-app:2.0`, que são imagens separadas. O Dockerfile não precisa mudar.

Parte A: alterar o `app.js`

Faça duas mudanças.

1. **Troque o texto da resposta** para indicar a versão 2.0.
2. **Faça o servidor escrever uma mensagem no console ao ligar.** O `.listen()` aceita um segundo argumento: uma função que o Node executa assim que o servidor está pronto. O `console.log(...)` escreve na saída, que é o que o `docker logs` mostra.

```
const http = require('http');

const servidor = http.createServer((req, res) => {
    res.end('Renatao - versao 2.0');
});

servidor.listen(3000, () => {
    console.log('Servidor v2.0 rodando na porta 3000');
});
```

Parte B: construir a imagem 2.0

```
docker build -t aluno-app:2.0 .
```

É o mesmo comando do Passo 4, só com a tag trocada. Confirme com `docker images` que a `aluno-app:2.0` apareceu.

Isto vem antes do `docker run`. Quem cria a imagem é o `build`, e o `run` só usa imagens que já existem.

Parte C: rodar a 2.0 em outra porta

O contêiner da 1.0 continua rodando e ocupa a porta 3000 do seu PC. Duas coisas não podem usar a mesma porta ao mesmo tempo. A solução mais didática é rodar a 2.0 na porta 3001 do PC, mantendo a 3000 dentro do contêiner, e dar a ele um nome diferente:

```
docker run -d --name container-devops2.0 -p 3001:3000 aluno-app:2.0
```

Só o número da **esquerda** mudou, porque dentro do contêiner o app continua escutando na 3000. A outra opção é parar o antigo com `docker stop aluno-app-container` e usar a 3000.

Parte D: conferir

1. `http://localhost:3001` deve mostrar o texto da 2.0.
2. `http://localhost:3000` continua mostrando o da 1.0, porque os dois contêineres rodam ao mesmo tempo.
3. `docker ps` mostra os dois ativos.
4. `docker logs container-devops2.0` mostra a mensagem do `console.log`.
5. `docker images` mostra o `aluno-app` com as tags `1.0` e `2.0`.

Erro 6: `pull access denied`

```
docker run -d --name container-devops2.0 -p 3000:3000 aluno-app:2.0
Unable to find image 'aluno-app:2.0' locally
docker: Error response from daemon: pull access denied for aluno-app,
repository does not exist or may require 'docker login'
```

Causa. O `docker run` foi executado **antes** do build da 2.0, então a imagem não existia. O Docker procurou no seu PC, não achou, foi procurar no Docker Hub e também não achou nenhuma imagem com esse nome.

Correção. Faça o `docker build -t aluno-app:2.0 .` primeiro e confirme com `docker images`. Só depois rode o `docker run`.

Erro 7: `port is already allocated`

```
docker: Error response from daemon: failed to set up container networking:
driver failed programming external connectivity on endpoint container-
devops2.0: Bind for 0.0.0.0:3000 failed: port is already allocated
```

Causa. O contêiner da versão 1.0 já usa a porta 3000 do PC, e o novo tentou usar a mesma.

Correção. Troque o número da esquerda do `-p` (por exemplo, `-p 3001:3000`) ou pare o contêiner antigo com `docker stop`.

Erro 8: o contêiner que ficou preso em `Created`

No erro anterior, o Docker imprimiu o ID do contêiner antes da mensagem de erro. Isso significa que ele **criou** o contêiner, mas não conseguiu **iniciá-lo**. O nome `container-devops2.0` continua ocupado, e rodar de novo com o mesmo nome dá o erro `name is already in use`.

Veja o contêiner parado, com status `Created`:

```
docker ps -a
```

Correção. Remova o contêiner preso e rode de novo com a porta certa:

```
docker rm container-devops2.0
docker run -d --name container-devops2.0 -p 3001:3000 aluno-app:2.0
```

O `docker rm` apaga o **contêiner**, nunca a imagem `aluno-app:2.0`, que continua intacta.

Resumo dos erros

Oito erros apareceram ao longo da atividade, e quase todos vêm de misturar nomes ou da ordem dos passos. Use a tabela para achar rápido o seu.

#	Mensagem	Causa	Correção
1	`failed to connect to the docker API`	Docker Desktop fechado; o motor está desligado	Abrir o Docker Desktop, esperar o motor subir e confirmar com `docker info`
2	Nenhuma mensagem, mas o nome da imagem fica errado	`-t node:20-alpine` usa o nome da imagem base	`docker build -t aluno-app:1.0 .`
3	`invalid reference format`	`node:20-alpine:1.0` tem dois `:`	Uma imagem tem um só `nome:tag`: `aluno-app:1.0`
4	`Unable to find image 'node:20-alpine' locally`	Rodou a imagem base em vez da sua	Rodar `aluno-app:1.0`

#	Mensagem	Causa	Correção
5	`invalid container name`	`:` no nome do contêiner	Nome inventado, sem `:`, como `aluno-app-container`
6	`pull access denied for aluno-app`	`docker run` antes do `docker build` da 2.0	Fazer o build e conferir em `docker images`
7	`port is already allocated`	A 1.0 já ocupa a porta 3000 do PC	Mudar a porta da esquerda (`-p 3001:3000`) ou parar a 1.0
8	`name is already in use`	Contêiner ficou em `Created` após o erro 7	`docker rm container-devops2.0` e rodar de novo

Regra para não se perder: imagem tem `nome:versão`; contêiner tem um nome livre, sem versão; `build` cria a imagem, `run` só usa imagem que já existe.

Cola de comandos e checklist final

Comandos usados na atividade

Comando	O que faz
`docker --version`	Mostra a versão do Docker instalada
`docker info`	Confirma que o motor está rodando
`docker build -t aluno-app:1.0 .`	Constrói a imagem a partir do Dockerfile da pasta atual
`docker images`	Lista as imagens do computador
`docker run -d --name NOME -p PC:CONTEINER IMAGEM:TAG`	Cria e inicia um contêiner em segundo plano com a porta publicada
`docker ps`	Lista os contêineres em execução
`docker ps -a`	Lista todos, inclusive os parados
`docker logs NOME`	Mostra a saída do app dentro do contêiner
`docker stop NOME`	Para um contêiner em execução
`docker rm NOME`	Apaga um contêiner parado (a imagem fica)

Comando	O que faz
`docker rmi IMAGEM:TAG`	Apaga uma imagem (só se nenhum contêiner a usa)

Para recomeçar do zero, caso queira refazer tudo para treinar:

```
docker stop aluno-app-container container-devops2.0
docker rm aluno-app-container container-devops2.0
docker rmi aluno-app:1.0 aluno-app:2.0
```

Checklist de entrega

- [ ] `app.js` com a mensagem da versão 2.0 e o `console.log`
- [ ] `Dockerfile` com `FROM`, `WORKDIR`, `COPY`, `EXPOSE` e `CMD`
- [ ] `.dockerignore` criado
- [ ] Imagem `aluno-app:1.0` construída
- [ ] Imagem `aluno-app:2.0` construída
- [ ] Contêiner da 1.0 respondendo em `localhost:3000`
- [ ] Contêiner da 2.0 respondendo em `localhost:3001`
- [ ] Prints de `docker ps`, `docker logs` e `docker images`
- [ ] Respostas da parte teórica no `README.md`
- [ ] Cabeçalho do PDF preenchido (aluno, turma, data)